



Computer Games Development CW208

Technical Design Document

Year IV

Jakub Stepień
C00284361

Jakub Stepień
C00284361

13/04/2026

Contents

Technical Design	3
Software	4
System Architecture	4
Player	4
Tile	9
Inventory	11
Pickup	13
UI	14
Level Editor	15
Game Flow	15
Controls	15
Future Improvements	16
References	16

Technical Design

Software

Backtrack is made entirely using the C++ programming language along with the SFML 3.0 library using Visual Studio as the IDE.

System Architecture

The main systems in Backtrack are:

- Player
- Tile
- Inventory
- Pickup
- UI
- Level Editor

Player

The player is probably the most complex system in this project, as it handles and deals with a lot of the logic of the game.

Classes/Structs

Other classes/structs that are important to the player class.

Enum class PlayerState:

An enum class to determine the current state of the player, it consists of the following states:

- Idle
- Running
- Jumping
- Falling
- Dashing
- Dying
- Reviving

Enum class CollisionSide:

An enum class to tell which side of a tile the player is colliding with, consists of:

- Top
- Bottom
- Left
- Right

Struct CollisionResult:

A struct to return when a collision occurs and the side the collision occurred, the struct consists of:

- Bool collided
- CollisionSide side

Variables

Variables that are used within the player class:

- sf::Texture m_texture
Texture used to hold the player spritesheet.
- sf::Sprite m_sprite
Sprite used to render the player.
- sf::Texture m_reviveTexture
Texture used to hold the revive spritesheet.
- sf::Texture m_heartTexture
Texture used to hold the heart spritesheet.
- std::vector<sf::Sprite> m_heartSprites
A vector used to hold the sprites for the health
- sf::RectangleShape m_hitbox
A rectangle shape around the player that is used for collision with the world.
- int m_maxHealth
An integer to keep track of the player's maximum health.
- int m_health
An integer to keep track of the player's current health.
- int m_deathCount
An integer to keep track of how many times the player has died.
- sf::Vector2f m_levelStartPosition
A Vector2f that keeps track of the current level's start point.
- sf::Vector2f m_position
A Vector2f that keeps track of the player's hitbox's current position on the level.
- sf::Vector2f m_spritePosition

A Vector2f that keeps track of the player's sprite's current position (slightly offset from m_position)

- sf::Vector2f m_velocity
A Vector2f that keeps track of the player's velocity.
- sf::Vector2f m_speed
A Vector2f that keeps track of the player's speed (x and y values are different because of sideways movement and jumping)
- Inventory m_inventory
Player's inventory object.
- bool m_hasDoubleJump
A bool to check if the player has unlocked double jump.
- bool m_doubleJumpReady
A bool to check if the player is able to double jump (A flag to prevent multiple jumps at a time).
- bool m_hasDash
A bool to check if the player has unlocked dashing.
- bool m_dashReady
A bool to check if the player is able to dash (A flag to prevent dashing multiple times in a row).
- sf::Clock m_dashDurationClock
Timer for how long the dash should be.
- sf::Time m_dashDuration
The amount of time in seconds the dash is.
- sf::Clock m_dashCooldownClock
Timer for how long the dash cooldown should be
- sf::Time m_dashCooldown
The amount of time in seconds the cooldown is.
- bool m_onGround
A bool to check if the player is on the ground.
- PlayerState m_previousState
Keeps track of the state from the last frame, used to transition between animations.

- `PlayerState m_playerState`
Keeps track of the current player state.
- `sf::Vector2i m_frameSize`
A `Vector2i` that keeps track of the size a frame from the spritesheet.
- `std::vector<sf::IntRect> m_playerFrames`
A vector that holds `IntRects` to be used as the animation frames.
- `sf::Time m_playerTime`
The amount of time between frames in the animation.
- `sf::Clock m_playerClock`
The timer used for animations.
- `int m_currentPlayerFrame`
An integer used as an index of the current frame in `m_playerFrames`.

Methods

The methods of the player class (not including getter methods):

- `Void Update()`
Mainly used to loop through animations and updating the health visual, but also prevents the player from moving left off-screen.
- `Void Render()`
Used to render the player sprite and heart sprites on the game window.
- `Void CheckInput()`
This is where most of the logic for the player is. This method handles most of the state transitions and key inputs from the player. For example, if A or D is pressed it changes the value of the player's x velocity to move left or right, and when the W key is pressed it changes the value for the player's y velocity to jump. Based on the velocity it sets the player's state which then in turn sets the correct frames to animate. Also sets the velocity of the player.
- `Void CheckState()`
Prints the current state to the console (Debug).
- `Void ApplyVelocityX`
Applies the x velocity from `checkInput()` to the player's position.

- **Void ApplyVelocityY**
Applies the y velocity from checkInput() to the player's position.
- **Void CheckCollisionX()**
Checks if there is a collision, then checks if the collision is on the left or right to set the player's position back away from that side of the tile to prevent clipping. If the tile is harmful, knockback the player and deal 1 heart of damage.
- **Void CheckCollisionY()**
Checks if there is a collision, the checks if the collision is on top or bottom to set the player's position back away from that side of the tile to prevent clipping, also if it's on top set the m_onGround bool to true and set m_doubleJumpReady to true. If the tile is harmful, knockback the player and deal 1 heart of damage.
- **CollisionResult getCollisionSide()**
The main collision detection system using AABB (Axis-Aligned Bounding Boxes). Checks if the player's hitbox overlaps with the tile's bounding box, if it does get the size of the overlap. If the overlap on the X axis is smaller than the Y axis it is a left/right side collision. If the overlan on the Y axis is smaller, it is a top/bottom side collision. Then return the CollisionResult.
- **Void CheckNewItem()**
Gets the item count from the inventory, and based on which item it is, it gives the effect, ex. 1st is increased speed, so if its the first item in the inventory, increase m_speed.x.
- **Void ResetHealth()**
Sets player's current health back to maximum health.
- **Void UpdateHearts()**
Updates the heart sprites based on the players current health. Loop the amount of time of maximum health, if the current iterated heart is less than the current health, set it to be the sprite of a full heart, if the iteration is past the current health, set it to be the sprite of an empty heart.
- **Void TakeDamage()**
If the player's health isn't 0, take away 1 from current health. If the current health is 0, set player state to dying, set velocity to 0 and increase the death count.
- **Void Animate()**
Check if the vector of frames is empty, if it has frames in it, check if the time per frame has passed, if it has then check if the next frame would be outside of

the vector, if not increase current frame by 1, if it would be outside, set current frame back to 0. Then set the sprite's texture rect to the new frame.

- **Void PlayAnimationOnce()**
Same as `Animate()` except no looping, when it reaches the last frame don't set current frame back to 0. Also if it reaches the last frame of the dying state, set state to reviving, and if it's the last frame in reviving, set to idle.
- **Void AddFrame()**
Pass in an `IntRect` and push it into the `m_playerFrames` vector.
- **Void SetFrames()**
Clears the player frame vector, and goes through a switch statement with all the player states, each state calls `addFrame()` for all the frames needed for that state animation.

Tile

The tile class is one of the more important classes in the game as it ties in with the player for a platforming game.

Classes/Structs

Other structs used by the tile class.

Struct `SurroundingTiles`:

A struct that holds 8 integers for each of the 8 surrounding tiles around a tile, consists of:

- `Int left`
- `Int right`
- `Int top`
- `Int bottom`
- `Int topLeft`
- `Int topRight`
- `Int bottomLeft`
- `Int bottomRight`

Variables

Variables used within the tile class:

- `sf::Vector2f position`
A `Vector2f` to hold the position of the tile.
- `sf::Texture* texture`
A pointer to the texture in the overall game class (1 texture for all tiles instead of having copies for each).

- `std::optional<sf::Sprite> sprite`
Sprite used to render the tile on screen (using `std::optional` because in SFML 3.0 sprites need to have a texture allocated at construction)
- `sf::IntRect tileRect`
IntRect for the single tile from the tilesheet spritesheet.
- `sf::RectangleShape rectangle`
A rectangle shape used as the hitbox for collisions with the player.
- `bool harmful`
A bool to check if the tile deals damage to the player (flag used for spike tiles).
- `bool outlineVisible`
A bool to check if the outline of the tile should be visible (flag used for rendering outline in level editor).
- `bool empty`
A bool to check if the tile should be empty (flag used for rendering only occupied tiles).

Methods

Methods used in the tile class (not including getter methods):

- `Void render()`
Used to render the tile sprites (and outlines in level editor).
- `Void setTile()`
Passes in an integer for tile type, and that tile's surrounding tiles, sets flags (empty, harmful), and sets texture intRect based on the tileType.
- `Void setPosition()`
Sets the tiles position, then sets the position of the hitbox and sprite to that position.
- `Void passTexture()`
Used to pass the texture from the overall game class when creating tiles.
- `Void setTileRect()`
Used for auto-tiling mainly, checks the tile type passed in and checks it against all of the surrounding tiles using many if statements, then sets the tile intRect to match the correct tile to the surrounding ones.

- `Bool isMouseOver()`
Bool that returns true when the mouse is hovering over a tile in the level editor.
- `Void showOutline()`
Used to set outline flag for rendering outlines in level editor.
- `Float distanceToTile()`
Passes is player position to get the distance from the player to the tile (used before collision checking to avoid checking impossible collisions).
- `Void setOutlineColour()`
Used primarily to change the outline colour for the outer edges of the level (since outer edges are off screen, easier to tell in editor when they are emphasized).

Inventory

The inventory class is another important class to keep track of player items and to display their information.

Classes/Structs

Other structs used by the inventory class:

Struct Item:

A struct that holds information for the item, consists of:

- String name
- String description
- `Sf::Texture* texture;`
- `Sf::Sprite sprite`

Variables

- `int m_itemCount`
An integer that keeps track of how many items are in the inventory
- `sf::RectangleShape m_inventoryBackground`
A semi-transparent black rectangle shape use as a vignette.
- `sf::Texture m_inventoryTexture`
Texture holding the inventory image
- `sf::Sprite m_inventorySprite`

Sprite to render inventory on screen.

- `sf::Texture m_highlightTexture`
Texture holding item highlight image.
- `sf::Sprite m_highlightSprite`
Sprite to render item highlight on screen.
- `sf::Vector2f m_position`
Vector2f to keep track of inventory position on screen (set to center of camera viewpoint).
- `std::vector<Item> m_items`
A vector to hold the items picked up by the player.
- `sf::Vector2f m_highlightPos`
Vector2f to keep track of item highlight position on screen.
- `sf::Vector2f m_itemStartPos`
Vector2f to keep track of the first item position on screen.
- `float m_itemOffset`
Float used to offset items after the first to place them in the right positions.
- `bool m_hoveringItem`
Bool checking if an item is hovered over with the mouse.
- `sf::Font m_font`
Font used for item information text.
- `sf::Text m_itemNameText`
Text used to render item name.
- `sf::Text m_itemDescriptionText`
Text used to render item description.
- `sf::Vector2f m_textPosition`
Vector2f used to track item info position on screen.

Methods

Methods used in the inventory class (not including getters):

- `Void update()`

Used to check the mouse position and check if an item is hovered over. If an item is hovered, highlight the item and show the info of the item.

- `Void render()`
Used to render the inventory sprite, background, items, and item info. Also takes in player hearts as an argument and render them as a part of the inventory.
- `Void addItem()`
Increase item count by 1, takes in a texture pointer as an argument from the item that was picked up, then based on which number item it is, it sets the item's information, and adds it to the items vector.
- `Void centreOnScreen()`
Takes the camera view position as an argument and sets the position of all inventory related items to center around that position.

Pickup

Variables

Variables used in the pickup/item class:

- `sf::RectangleShape m_shape`
Rectangle shape used as hitbox for collision with the player.
- `sf::Texture* m_texture`
Texture pointer that points to item texture from game class.
- `std::optional<sf::Sprite> m_sprite`
Sprite used to render the tile on screen (using `std::optional` because in SFML 3.0 sprites need to have a texture allocated at construction)
- `sf::Vector2f m_position`
Vector2f used to track the position of the item on screen.
- `float bobbingSpeed`
Float used to track speed of the item floating up and down.
- `float bobbingHeight`
Float used to track how high the item floats up and down.

Methods

Methods used in the pickup/item class (not including getter methods):

- `Void update()`
Used to calculate the new Y value for the floating animation of the item using a sine wave function. Then the sprite's Y position is updated.
- `Void render()`
Used to render the item sprite
- `Void passTexture()`
Used to pass texture from game class to be able to make different items using the same class.
- `Void setPosition()`
Used to set the position of the item in the level.
- `Bool collidesWithPlayer()`
Checks the distance from the player to the item, if it is less than 50 pixels, return true.

UI

The UI can be split into a few separate portions, mainly:

- Menu
- Hud
- Inventory
- Level Editor
- End Screen

Menu

The menu consists of a background, a title text, and some buttons. The buttons are “Play”, “Level Editor”, “Controls” and “Exit”. When a button is hovered the text scales up, and an outline shows up around the text. This is made to clearly emphasize which button the player is currently selecting. If play is selected, the game state goes to gameplay, if level editor is selected, the game will swap to the editor screen, if controls is selected, the player will see the screen with all the controls and if exit is selected the game closes.

Hud

The hud is very simple, consisting only of heart sprites that show in the top-left corner of the screen during gameplay. The heart sprites show how much health the player is currently on as a full heart represents 1 health.

Inventory

A more complex UI, this shows the player their health, any items they own, and the items can be hovered to further show any information about them. We know which item is clearly hovered because the item highlight appears around the item.

Level Editor

The level editor shows the grid of tiles but with a visible outline, this lets the player easily see which tile they are selecting and placing on the grid.

End Screen

The end screen that shows up when the player finishes the game, shows the player's completion time and total amount of deaths. Some players like being able to see statistics like these and will try more attempts to beat their time or complete the game with a lower amount of deaths.

Level Editor

The level editor is an easy way of visualising and making the levels for the future. I made the levels load from text files into a 2D array of integers to be able to have a few levels prepared. While in the level editor, pressing the "S" key will save the current level layout to a file called newLevel.txt. The level editor also includes an auto-tiling feature, which allows me to not worry about exactly what type of tile I want to place considering I have about 14 different grass tiles, placing all those separately could take a long time to look good. So instead, each tile placed checks all its surrounding tiles and matches them all accordingly with the sprite sheet to look seamless.

Game Flow

When Backtrack is started, the player is met with a title screen. Pressing any button will lead the player onto a main menu where they can choose to go into gameplay or into the level editor. When the player goes into gameplay, they have 16 levels to beat to fully complete the game. They can check their inventory during gameplay. As the player progresses through the levels, they get increasingly more difficult. Picking up new items unlocks new abilities for the player which makes the player adapt to the new changes and use them to progress with the levels.

Controls

- A/D = Move left/move right respectively
- W = Jump
- Tab = Inventory

- Left Shift = Dash
- Escape = Main Menu

Future Improvements

Some improvements that I can already think of that could be beneficial for the game include:

- Using a resource holder to manage all the textures and sprites.
- Larger sized levels to make better designed/more challenging levels.
- More movement types to give the player a wider range of movement combinations.
- Making a visual menu in the level editor for selecting tiles.

References

Wenyao L.; Jinhua C.; Zemin L.; Rui F.; Tong H.; Lu D. (2025, March). Automatic tile position and orientation detection combining deep-learning and rule-based computer vision algorithms [Online]. (URL <https://www.sciencedirect.com/science/article/abs/pii/S092658052500041X>).

Yi-Si X.; Xiaoping P. L.; Shao-Ping X. (2010, June). Efficient collision detection based on AABB trees and sort algorithm [Online]. (URL <https://ieeexplore.ieee.org/document/5524093>).